

You are a senior Laravel + Inertia/Vue architect working on **UHMS — Ultimate Hospital Management System**.

The project user manual has been updated with the latest UHMS workflows. Your task is to align the actual codebase with the documented workflow.

Use the updated manual as the source of truth:

```
docs/UHMS_Updated_User_Manual.md
```

Also inspect the existing manual if present:

```
UHMS_USER_MANUAL.md
```

Your job is to compare the current implementation against the updated manual, identify gaps, and implement the required workflow corrections carefully.

Do not rewrite the whole project blindly. Work module by module, preserve working code, and avoid unrelated refactors.

1. Main Objective

Align the UHMS codebase with the updated documented workflows:

1. One visit starts at triage.
 2. Triage completion sends the patient to waiting consultation.
 3. Doctor must click `Start Consultation` before entering clinical data.
 4. One visit has one main invoice.
 5. Every department adds billable items to the same visit invoice.
 6. Billing must use consistent invoice item calculations.
 7. Insurance pricing must be applied consistently everywhere.
 8. Insurance covered is not payment.
 9. Discount is manually entered by authorized users.
 10. Paid amount only comes from real payments.
 11. Prescribing does not reduce stock.
 12. Dispensing reduces stock.
 13. Stock movements are the source of truth.
 14. Stock balances are only cached current quantity.
 15. Optional modules must not break the core visit workflow.
-

2. First Step: Audit the Current Codebase

Before implementing anything, inspect the current project.

Check:

- routes
- controllers
- models
- migrations
- services
- Vue/Inertia pages
- layouts
- sidebar/module logic
- billing implementation
- inventory implementation
- pharmacy workflow
- investigation workflow
- consultation workflow
- triage workflow
- documentation files

Create an audit note listing:

```
Implemented correctly
Partially implemented
Missing
Broken
Legacy/unused logic still present
```

Do not start coding until you understand what already exists.

3. Required Architecture Rules

Use this architecture:

```
Controllers
↓
Form Requests
↓
Services
↓
Models / Repositories
↓
Events / Jobs / Notifications
```

Controllers must remain thin.

Business logic must be placed in services.

Required services include or should be created/updated:

```
VisitWorkflowService
TriageService
ConsultationService
BillingService
InvoiceService
PaymentService
InsuranceService
ServicePricingService
PrescriptionService
InvestigationRequestService
InvestigationCatalogueService
InvestigationResultService
PharmacyDispensingService
StockMovementService
StockBalanceService
StockTransferService
StockAdjustmentService
StockReturnService
ModuleService
```

Do not put workflow, billing, insurance, stock, or investigation rules directly in controllers or Vue components.

4. Visit and Triage Workflow

Documented Rule

Every visit starts at triage.

When a visit is created:

```
status = TRIAGE
```

When triage is saved:

```
TRIAGE → WAITING_CONSULTATION
```

Implementation Requirements

Ensure visit creation:

1. creates the visit
2. selects valid insurance or Cash and Carry
3. creates/gets the main visit invoice
4. adds initial selected services to the invoice if applicable
5. sends patient to triage

Ensure triage save:

1. saves vitals
2. calculates triage score
3. stores triage score
4. changes visit status to `WAITING_CONSULTATION`
5. logs the transition
6. updates queue without full page reload

All status changes must go through:

VisitWorkflowService

Do not update visit status directly from controllers or Vue.

5. Consultation Workflow

Documented Rule

Doctor must click:

Start Consultation

before entering clinical data.

This changes status:

`WAITING_CONSULTATION` → `CONSULTING`

If already started, show:

Continue Consultation

Implementation Requirements

On consultation queue:

- show only visits with:
 - WAITING_CONSULTATION
 - CONSULTING

Action button logic:

```
WAITING_CONSULTATION → Start Consultation  
CONSULTING → Continue Consultation
```

On consultation page:

- disable complaints, diagnosis, prescriptions, investigations, treatment, and notes until consultation is started
- show clear message: Click Start Consultation to begin entering clinical information.
- preserve active tab after saving
- avoid full page reloads

Use:

```
VisitWorkflowService::startConsultation(...)  
ConsultationService
```

6. Diagnosis Workflow

Implement or verify:

- diagnosis can be provisional or final
- first diagnosis becomes primary by default
- user can set another diagnosis as primary
- only one primary diagnosis per visit
- diagnosis type can be edited if allowed

Do not reset the consultation page after saving diagnosis.

7. Prescription Workflow

Documented Rule

Prescribing does not reduce stock.

Stock is reduced only when pharmacy dispenses the drug.

Implementation Requirements

Fix/verify prescription creation:

- prescription saves correctly
- linked to visit
- linked to doctor
- linked to patient if needed
- validation errors display properly
- no silent 302 redirect
- prescription list updates without full reload
- adding multiple drugs works where allowed

Do not create stock movements during prescription creation.

Use:

PrescriptionService

8. Investigation Workflow

Investigations are not limited to Lab.

Investigation departments may include:

Lab
X-ray
Scan
CT-scan
Ultrasound
ECG
any configured investigation-type department

Doctor Request Flow

On consultation page:

1. doctor selects investigation department
2. system loads services under that department
3. doctor selects one or more services
4. system creates investigation request
5. request appears on the correct investigation department request page

Display Rule

On consultation page, requested investigations must be grouped by department.

Example:

```
LAB
- Full Blood Count - Pending
- Malaria Test - Done

X-RAY
- Chest X-Ray - Accepted
```

Doctors must be able to view results from the consultation page.

Doctors must not be able to delete an investigation item after results have been entered.

Use:

```
InvestigationRequestService
InvestigationResultService
BillingService
```

9. Investigation Request Acceptance

Documented Rule

Investigation staff must select requested items before accepting.

Only selected accepted items are billed.

Implementation Requirements

On investigation request view page:

- show requested items in a table
- each item has checkbox/select option
- Accept button disabled until at least one item is selected
- clicking Accept:
 1. validates selected items
 2. accepts only selected items
 3. bills only selected items
 4. adds invoice items to the visit's main invoice
 5. does not bill unselected items
 6. updates request/item statuses

Do not create a separate invoice.

Use:

```
BillingService::addItemToVisitInvoice(...)
InvoiceService::getOrCreateVisitInvoice(...)
```

10. Investigation Catalogue

Documented Rule

Replace old concept:

Lab Test Catalogue

with:

Investigation Catalogue

The catalogue is based on services from investigation-type departments.

Correct structure:

```
Investigation Service
→ Optional Headers / Categories
→ Criteria
```

Do not create separate catalogue tests detached from services.

Implementation Requirements

Investigation Catalogue page should:

1. list services where department type is `investigation`
2. allow selecting a service
3. allow configuring headers/categories for that service
4. allow configuring criteria for that service
5. allow criteria to belong to a header or stand alone

Suggested models/tables:

```
investigation_headers  
investigation_criteria
```

Criteria must belong to a service.

Headers must belong to a service.

Use:

```
InvestigationCatalogueService
```

11. Investigation Results, Verification, and Printing

Result entry must load configured headers and criteria from Investigation Catalogue.

When result is entered:

- save values against criteria
- preserve unit/reference range snapshot
- update item status
- allow View Result
- allow Verify Result for authorized users
- allow Print only after verification unless system setting allows otherwise

Doctors must be able to view results from the consultation page.

Use:

```
InvestigationResultService
```

Do not save results only as unstructured text.

12. Billing Model

Documented Rule

One Visit = One Main Invoice

All billable activities during a visit add items to the same invoice.

Examples:

- consultation service
- investigation item
- pharmacy item
- ward charge
- procedure
- scan
- x-ray

Do not create multiple active invoices for the same visit.

Implementation Requirements

Ensure:

- invoice is created automatically when visit starts or first billable item is added
- only one active invoice exists per visit
- every billable department uses the same billing service
- no controller creates invoice items directly

Use:

```
InvoiceService::getOrCreateVisitInvoice(...)  
BillingService::addItemToVisitInvoice(...)
```

13. Invoice Item Calculation Rules

Update invoice item logic to match the manual.

Required fields:

```
cash_price
insurance_price
selected_price
quantity
discount_amount
insurance_covered
patient_payable
paid_amount
balance
payment_status
```

Formulas

```
insurance_covered = cash_price - insurance_price
```

If no insurance applies:

```
insurance_covered = 0
```

```
patient_payable = (selected_price * quantity) - discount_amount
```

```
balance = patient_payable - paid_amount
```

Important:

- `insurance_covered` is not a payment
- `insurance_covered` must not be assigned to `paid_amount`
- `discount_amount` is manually entered
- discount must not equal insurance covered automatically
- `paid_amount` only comes from real payments/payment allocations

Remove or stop using legacy fields such as:

```
unit_price
is_nhis_covered
nhis_approved_amount
approved_amount
coverage_percentage
covered_amount
insurance_paid
insurance_payment
relevance
```

Use `selected_price` in views instead of `unit_price`.

14. Insurance Pricing Rules

Insurance must be applied consistently everywhere.

Pricing priority:

```
Provider-specific insurance service price
↓
General insurance type service price
↓
Base price / Cash and Carry price
```

Rules:

- Cash and Carry uses service base price.
- NHIS is not special; it is one insurance type.
- Provider-specific prices override general insurance type prices.
- Coverage percentage must not be used in price calculation.
- If insurance is invalid/expired/exhausted/missing, fallback to Cash and Carry.
- Store CCC code where applicable, preferably on patient insurance.

Use:

```
InsuranceService
ServicePricingService
```

Every department must respect the patient's selected visit insurance through the central billing/pricing services.

15. Payments and Payment Allocation

Patients can pay during an ongoing visit.

Payments may clear:

- full invoice
- selected invoice items
- part of selected invoice items

Implement or verify:

```
payments
payment_allocations
```

Payment allocation must update:

- item paid amount
- item balance
- item payment status
- invoice totals
- invoice status

Do not allow payment to affect stock.

Use:

```
PaymentService
InvoiceService
```

16. Manual Discounts

Discounts are entered manually from the Invoice View page.

Implementation requirements:

- add discount input or Apply Discount modal per invoice item
- only authorized users can apply discounts
- discount cannot be negative
- discount cannot exceed `selected_price * quantity`
- discount recalculates:
 - patient payable
 - balance
 - payment status
 - invoice totals

Use:

```
BillingService::applyDiscount(...)
```

Do not calculate discount only in Vue.

Backend must be authoritative.

17. Invoice View UI

Invoice view should show simplified, accurate data.

Recommended columns:

```
Service / Description
Selected Price
Discount
Patient Payable
Paid Amount
Balance
Status
Action
```

Remove or avoid showing:

```
Qty
Cash Price
unit_price
NHIS approved amount
coverage percentage
legacy relevance fields
```

If quantity and cash price are needed internally, keep them in the database but do not show them on the simplified invoice view.

18. Pharmacy and Dispensing

Documented Rule

- prescribing does not reduce stock
- dispensing reduces stock
- only dispensed drugs are billed
- dispensed drugs are added to the visit's main invoice
- dispensing creates a `PHARMACY_DISPENSED` OUT stock movement

Implementation requirements:

1. pharmacy can dispense multiple drugs where valid
2. no silent `302` redirects
3. validation errors display clearly
4. stock availability is checked from stock balance
5. billing uses `BillingService`
6. stock uses `StockMovementService`

Use:

```
PharmacyDispensingService  
BillingService  
StockMovementService  
StockBalanceService
```

19. Inventory and Stock Movement System

Documented Rule

```
stock_movements = source of truth  
stock_balances = fast current stock cache
```

Current stock:

```
Total IN movements - Total OUT movements
```

Do not manually overwrite product/drug quantity as current stock.

Required Movement Types

```
OPENING_STOCK  
PURCHASE_RECEIVED  
PHARMACY_DISPENSED  
TRANSFER_IN  
TRANSFER_OUT  
RETURN_IN  
RETURN_OUT  
ADJUSTMENT_IN  
ADJUSTMENT_OUT  
DAMAGED  
EXPIRED  
REVERSAL_IN  
REVERSAL_OUT
```

Implementation requirements:

- opening stock creates opening stock movement
- purchase receiving creates IN movement
- pharmacy dispensing creates OUT movement
- transfers create paired OUT and IN movements
- returns create return movements
- stock adjustments create adjustment movements
- damaged/expired stock creates OUT movements
- corrections use reversal movements, not deletion

Use:

```
StockMovementService
StockBalanceService
StockTransferService
StockAdjustmentService
StockReturnService
```

20. Purchase Orders and Receiving

Documented Rule

Creating a purchase order does not increase stock.

Stock increases only when purchase items are received.

Implementation requirements:

- fix error: `at least one item is required` even when items are selected
- ensure selected items are sent to backend under the expected key, preferably `items`
- validation errors display properly
- purchase order saves with multiple items
- receiving purchase items creates `PURCHASE_RECEIVED` stock movement
- receiving updates stock balance
- purchase order status supports:
 - pending
 - partially received
 - received
 - cancelled

Do not increase stock when purchase order is merely created unless the workflow explicitly says received immediately.

21. Stock Transfers

Transfers move stock from one location to another.

A completed transfer must create:

TRANSFER_OUT from source location
TRANSFER_IN into destination location

Rules:

- source and destination must be different
- quantity must be greater than zero
- source must have enough stock
- both movements must link to the same transfer record

22. Stock Returns

Returns must create stock movements.

RETURN_IN = stock comes back into a location
RETURN_OUT = stock leaves a location, e.g. supplier return

Do not delete original stock movements.

23. Stock Adjustments, Damaged, Expired, and Reversals

Stock adjustment is for physical count corrections.

Adjustment types:

ADJUSTMENT_IN
ADJUSTMENT_OUT

Adjustment requires:

- product/drug
- location
- adjustment type
- quantity
- reason
- authorized user

Damaged stock:

DAMAGED + OUT

Expired stock:

```
EXPIRED + OUT
```

Corrections must use reversals:

```
REVERSAL_IN  
REVERSAL_OUT
```

Do not delete old stock movements.

24. Module System

Core modules should always remain active:

```
auth  
users/roles  
patients  
visits  
triage  
consultation  
departments  
services  
billing  
settings
```

Optional modules can be enabled/disabled:

```
insurance  
claims  
pharmacy  
inventory  
investigations  
analyzer  
HR  
reports  
notifications
```

If optional module is disabled:

- hide it from sidebar
- protect routes
- use safe fallback behavior

Examples:

- Insurance disabled → Cash and Carry is used
- Pharmacy disabled → prescriptions can be recorded but dispensing unavailable
- Analyzer disabled → investigation results entered manually

Fix any issue where modules are not loading.

Use:

```
ModuleService
```

25. SPA / Inertia / Vue Requirements

Critical pages must not perform unnecessary full page reloads:

- visit creation
- triage
- consultation
- prescriptions
- investigation requests
- result entry
- invoice view
- payment allocation
- pharmacy dispensing
- purchase orders
- stock movements

Requirements:

- preserve active tabs
- show validation errors in-page
- do not dismiss modals before successful response
- do not produce silent 302 redirects without visible errors
- use Inertia form errors properly
- keep UI state after validation failure

26. Reports

Ensure reports use the correct source records.

Billing reports should use:

```
invoices
invoice_items
payments
payment_allocations
```

Inventory reports should use:

```
stock_movements
stock_balances
```

Investigation reports should use:

```
investigation_requests
investigation_request_items
investigation_results
investigation_result_values
```

Do not report from legacy/removed tables like `visit_services` .

27. Legacy Cleanup

Search for and remove/replace active usage of legacy concepts:

```
visit_services
unit_price
coverage_percentage
is_nhis_covered
nhis_approved_amount
Lab Test Catalogue as primary concept
hard product quantity as current stock
separate invoices per department
insurance covered as paid amount
discount automatically derived from insurance
```

Do not drop database columns blindly until code no longer uses them.

Recommended cleanup approach:

1. stop writing to legacy fields
2. stop reading from legacy fields
3. migrate data if needed
4. update views
5. remove columns/tables only when safe

28. Testing and Verification

Add or update tests for:

Visit / Triage

- visit starts at triage
- triage save changes status to waiting consultation

Consultation

- cannot enter clinical data before start
- start consultation changes status to consulting
- prescription saves without stock reduction

Investigation

- doctor request appears in investigation queue
- only accepted selected items are billed
- result entry blocks doctor deletion
- doctor can view result

Billing

- one invoice per visit
- all departments add to same invoice
- insurance pricing is consistent
- insurance covered is not paid amount
- discount is manual
- balance = patient payable - paid amount

Payments

- payment allocation updates invoice item statuses
- partial payment works
- full payment works

Inventory

- purchase order creation does not affect stock
 - receiving purchase increases stock
 - dispensing decreases stock
 - transfer creates IN and OUT
 - adjustment works
 - reversal works
 - stock balance rebuild works
-

29. Deliverables

Provide:

1. Gap analysis against `docs/UHMS_Updated_User_Manual.md`
 2. Files modified
 3. New or updated migrations
 4. Updated models and relationships
 5. Updated services
 6. Updated Inertia/Vue pages
 7. Updated validation requests
 8. Updated tests or verification notes
 9. Confirmation that documented workflows match the implementation
 10. Any remaining TODOs or manual migration notes
-

30. Important Rules

Do not rewrite the whole system blindly.

Do not refactor unrelated modules.

Do not bypass services.

Do not create invoices directly in controllers.

Do not create invoice items outside `BillingService`.

Do not update visit status outside `VisitWorkflowService`.

Do not reduce stock on prescription creation.

Do not let payment affect stock.

Do not use product quantity as source of truth.

Do not treat NHIS as special.

Do not treat insurance covered as paid amount.

Do not auto-calculate discount from insurance.

Do not use `visit_services` for the active visit billing display.

Do not allow full page reloads where SPA behavior is expected.

Now inspect the current codebase, compare it with `docs/UHMS_Updated_User_Manual.md` , then implement the required changes module by module while preserving existing working functionality.